

Post-Quantum Cryptographic Analysis of Sigstore

Full version with proofs

Abstract

Sigstore signs millions of released artifacts in the Python, npm, Kubernetes and container ecosystems. A Sigstore signature is a conjunction of six authenticators, which are an identity token, a short lived certificate, a signed certificate timestamp, a timestamp token, a log inclusion proof and a signed checkpoint. All of the deployed schemes fall to Shor’s algorithm and the project has begun to experiment with post-quantum replacements, yet no formal model of Sigstore exists, so it is unknown which of its guarantees a change of algorithms preserves. A second gap lies in the primitives. The single use keys of a signer call for a one-time hash-based signature, but the tight proof for the Winternitz variant inside SLH-DSA covers only messages fixed before the public key.

In this work, we give the first provable-security treatment of Sigstore. We formalize it as a transparent keyless signing scheme, prove unforgeability from properties of its deployed building blocks, and introduce a harvest-now-forge-later model in which the adversary turns quantum at a time t_Q and must backdate a signature. We show that no substitution of post-quantum signatures suffices there, while a log root pinned before t_Q leaves the quantum adversary facing collision resistance alone. We then resolve the primitive gap with a tight proof of adaptive multi-instance one-time unforgeability in the quantum random oracle model. Guided by both results we design Latch, which replaces the six authenticators by one hash-based signature, one inclusion proof and one epoch signature. Against the direct algorithm substitution Latch reduces the bundle by 4.1×, log storage by 12.4× and verification time by 3.3×, and against the strongest alternative by 3.5×, 5.6× and 2.6×.

Keywords

software signing, Sigstore, transparency logs, post-quantum cryptography, hash-based signatures, quantum random oracle model

1 Introduction

Attacks through the software supply chain reach far more victims than attacks on a single target [27, 39]. Signing released artifacts is the standard defense, but classical code signing asks maintainers to manage long lived keys, and adoption has been poor for that reason [37, 50]. Sigstore removes this burden by issuing a short lived certificate that binds a fresh public key to an identity the signer already controls, such as an electronic mail address or a continuous integration workflow, and by publishing what it issues in append-only logs so that misissuance is detectable [37, 46, 47]. It follows the automated certificate authority model of Let’s Encrypt [1] and the transparency model of Certificate Transparency [28, 29], and it is now the signing layer of major package ecosystems.

Cryptographically, a Sigstore verification is a conjunction of six cryptographic checks over objects produced by the signer and five services, as Figure 1 shows. The signer obtains a token from an identity provider [41] and a short lived X.509 certificate for a fresh key from the Fulcio certificate authority [14, 46], which embeds a signed

certificate timestamp of a Certificate Transparency log [28]. It then signs the artifact, obtains an RFC 3161 timestamp token [2], and receives an inclusion proof and a signed checkpoint from the Rekor transparency service [47]. A verifier accepts only if all checks pass and the timestamp falls inside the certificate validity window, which is the hybrid model of certificate validation of Braun et al. [10, 49].

All six signatures are ECDSA, Ed25519 or RSA [25, 48], and Shor’s algorithm recovers each secret key from its public key [44]. The project experiments with ML-DSA and SLH-DSA and has published a policy on the post-quantum families it will consider [34, 35, 45], which names a concrete obstacle. Stateful hash-based schemes [15, 22, 30] fit the single use key of a signer, but not a certificate authority or a log, which would have to track per-signature state across a manual key rotation ceremony. What a change of algorithms buys is unanswered, because Sigstore has only an informal threat model [33, 37].

First problem. A Sigstore signing key is used once, so a one-time signature suffices for the signer, and the most conservative and compact candidate is WOTS-TW, the Winternitz variant inside SLH-DSA and XMSS [7, 21, 35]. Its tight proof by Hülsing and Kudinov [23] covers only non-adaptive attacks, in which the adversary commits to the signed message before it sees the public key. Barbosa et al. [3] call the adaptive case an open question, settle it for XMSS in the classical random oracle model, and leave the quantum setting as future work that would need adaptive reprogramming [19], and the later machine-checked treatment of SLH-DSA restates this limitation [4]. A Sigstore signer publishes its one-time public key in a certificate before it signs a digest that an adversary controlling a dependency can influence, so neither result applies.

Second problem. As we identify in this work, Sigstore has no formal model, and in particular none that separates the moment a signature is produced from the moment it is checked. A certificate is valid for ten minutes while the artifact it covers is installed for years, so without that separation one cannot ask whether a signature produced today resists an adversary that acquires a quantum computer tomorrow.

Contributions. We make three contributions.

- A tight proof of adaptive multi-instance one-time unforgeability in the quantum random oracle model for a randomized message variant of WOTS-TW, which resolves the first problem in the single use setting (Section 5).
- A formalization of Sigstore as a transparent keyless signing scheme with an unforgeability proof, and a harvest-now-forge-later notion in which the adversary becomes quantum at a time t_Q and must backdate a signature. No choice of post-quantum signatures achieves it, while a log root pinned before t_Q leaves the quantum adversary facing collision resistance alone (Section 4).
- Latch, which fuses certificate issuance, transparency, timestamping and inclusion into one authenticated structure and gives the signer a one-time hash-based key. We implement

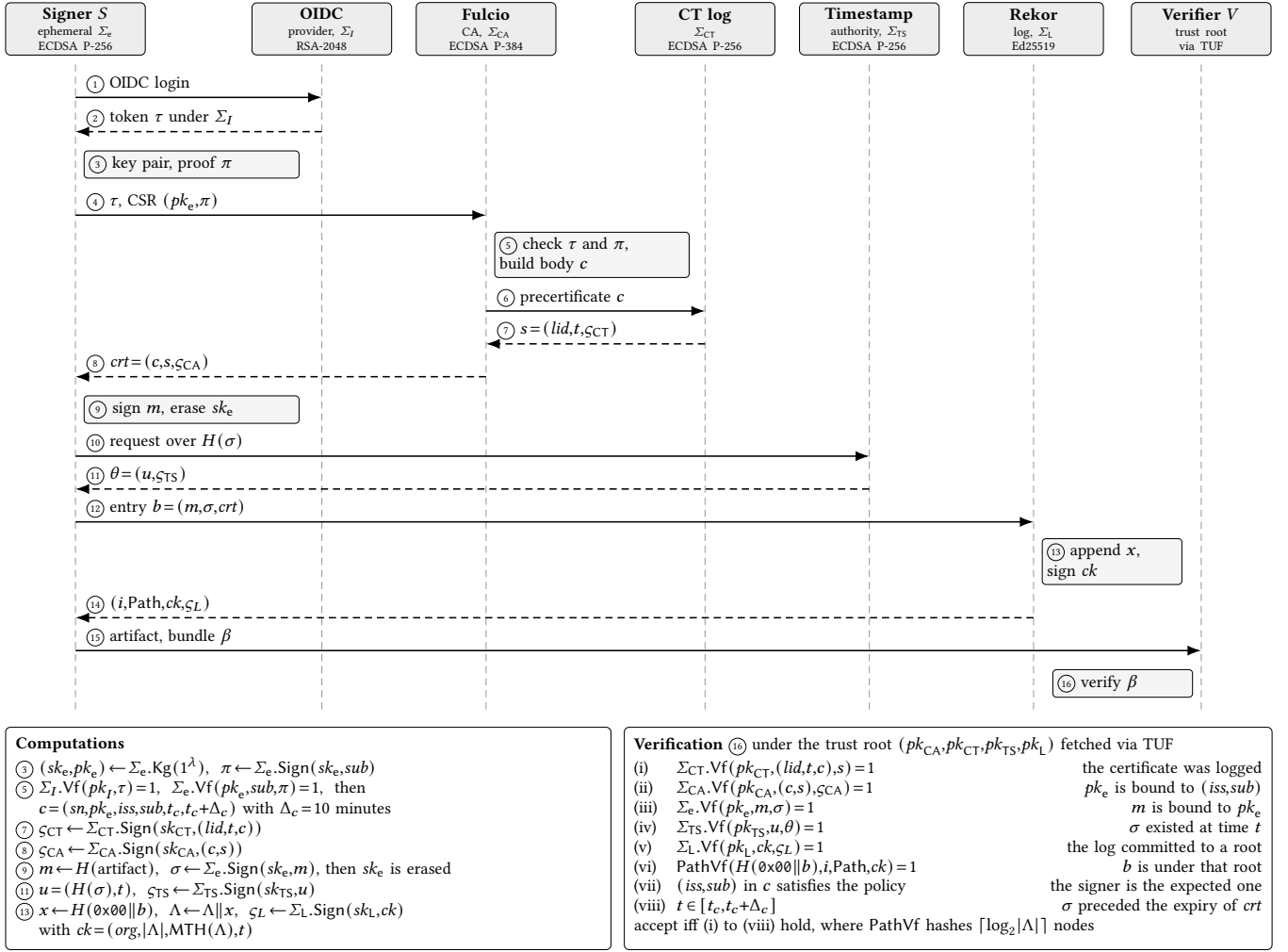


Figure 1: Sigstore signing and verification, with the deployed scheme of each party under its name. Solid arrows are requests, dashed arrows are responses, and shaded steps are local computations expanded in the lower panels. The identity of the signer S is the issuer and subject pair (iss, sub) , ζ_X is a signature under Σ_X , the certificate body c holds a serial number sn and a validity window starting at t_c , lid names the CT log and t is the time a service asserts, x is leaf i of the Rekord log state Λ whose checkpoint ck names the log origin org , and $\beta = (crt, \sigma, \theta, i, Path, ck, \zeta_L)$ is the bundle.

it and six alternatives, compare them by complexity and by measurement, and find Latch smallest and fastest on all axes (Sections 6 and 7).

Threat model. Before t_Q the identity provider, the issuing service and the log are honest, and the adversary may open accounts, submit arbitrary entries, read the public log and corrupt any identity other than its target. After t_Q it is quantum and holds the secret keys of a declared set of signature schemes, which models Shor recovery from the published public keys. Witness cosignatures, which stop a log from presenting different views to different parties [13], are identical in all constructions we compare and are left out of the cost accounting.

2 Preliminaries

We write $\{0, 1\}^n$ for bit strings of length n , $x \parallel y$ for concatenation and $x \leftarrow S$ for uniform sampling from a finite set S . Adversaries in the post-quantum statements are quantum polynomial time and get superposition access to public hash functions, modeled as quantum random oracles [8]; oracles holding secrets answer classical queries only, and q counts quantum oracle queries.

Signature schemes. A signature scheme $\Sigma = (Kg, Sign, Vf)$ has the usual syntax, and $\text{Adv}_{\Sigma}^{\text{euf}}(\mathcal{A})$ is the advantage in the existential unforgeability game under chosen message attack [18]. We write $\text{Adv}_{\Sigma}^{d\text{-euf1}}(\mathcal{A})$ for the multi-instance one-time variant, in which the adversary receives d public keys, asks one signing query per instance, and forges for one instance on a message not queried there. When

the d messages must be fixed before the public keys are given, we call the notion non-adaptive and write $\text{Adv}_{\Sigma}^{d\text{-na}}(\mathcal{A})$, following [23].

Tweakable hash functions. A tweakable hash function $\text{Th} : \mathcal{P} \times \mathcal{T} \times \{0,1\}^* \rightarrow \{0,1\}^{8n}$ takes a public parameter P , a tweak A and a message, and outputs n bytes [7]. We use the three properties the tight analysis of WOTS-TW needs, each with distinct tweaks and p targets, which are single-function multi-target collision resistance $\text{Adv}_{\text{Th},p}^{\text{tr}}$, preimage resistance $\text{Adv}_{\text{Th},p}^{\text{pre}}$ and undetectability $\text{Adv}_{\text{Th},p}^{\text{ud}}$ [23, 24], where distinct tweaks are what removes the multi-target speedup a plain hash would suffer. We instantiate $\text{Th}(P,A,M) = \text{trunc}_n(H(P\|A\|M))$ with H equal to SHA-256 and A a 22 byte address, which is the construction of the SHA-2 parameter sets of SLH-DSA [35].

Merkle trees. For a list of leaves we use the tree hash of RFC 6962, in which a leaf commits as $H(0x00\|x)$ and an internal node as $H(0x01\|\ell\|r)$, with the tree split at the largest power of two below its size [28, 29, 32]. We write $\text{MTH}(L)$ for the root of a leaf list L , $\text{Path}(L,i)$ for the inclusion proof of index i and PathVf for the verification procedure of RFC 9162. Soundness of inclusion proofs follows from collision resistance of H , whose generic quantum advantage after q queries is $O(q^3/2^{8n_h})$ for output length n_h bytes [9, 51].

Adaptive reprogramming. We use the tight bound of Grilo et al. [19]. If a quantum distinguisher issues q queries to a random oracle and R reprogramming instructions, each of which reprograms the oracle at a point whose first component is drawn uniformly from a set X_1 , then its distinguishing advantage between the reprogrammed and the original oracle is at most $\frac{3R}{2}\sqrt{q/|X_1|}$. We also use the generic quantum bound for finding a marked input of density μ , which after q queries is at most $8\mu(q+1)^2$ [24].

3 Sigstore as a Cryptographic Scheme

3.1 Syntax

We first fix a syntax that covers Sigstore and the constructions we compare with it. An identity is a pair $id = (iss, sub)$ of an issuer name and a subject, because a subject means nothing without the provider that vouches for it [46]. A verification policy names the identity that is allowed to sign a given artifact.

Definition 3.1 (Transparent keyless signing scheme). A transparent keyless signing scheme for an identity space $\mathcal{I}$ and a message space $\mathcal{M}$ is a tuple $\Pi = (\text{Setup}, \text{Auth}, \text{Prep}, \text{Issue}, \text{Asm}, \text{Vf})$ of algorithms, together with a log state Λ and its public view $\text{View}(\Lambda)$:

- $\text{Setup}(1^\lambda) \rightarrow (pp, sk_S)$ outputs public parameters pp , which contain the trust root, and the secret keys sk_S of the services;
- $\text{Auth}(sk_I, id) \rightarrow \tau$ is run by the identity provider and outputs an identity token τ ;
- $\text{Prep}(pp, id, m) \rightarrow (\rho, st)$ is the local computation of the signer on a message m , producing a request ρ and a private state st ;
- $\text{Issue}(sk_S, \Lambda, \tau, \rho) \rightarrow (\Lambda', \alpha)$ is run by the services, which check the token and the request, update the log state, and return issuance material α ;
- $\text{Asm}(st, \alpha) \rightarrow \beta$ assembles the verification bundle β ;
- $\text{Vf}(ts, id, m, \beta) \rightarrow \{0,1\}$ checks β against a verifier trust state ts , an expected identity and an expected message.

```

SS.Prep(pp, id, m)
  (sk_e, pk_e) ← Σ_e.Kg(1^λ)
  π ← Σ_e.Sign(sk_e, sub) / possession
  σ ← Σ_e.Sign(sk_e, m) / artifact
  return (ρ = (pk_e, π, σ), st = ρ)
    
```

```

SS.Issue(sk_S, Λ, τ, ρ)
  if Σ_I.Vf(pk_I, τ) = 0 then return ⊥
  (iss, sub) ← τ; if Σ_e.Vf(pk_e, sub, π) = 0 then return ⊥
  c ← (sn, pk_e, iss, sub, t, t + Δ_c) / to be signed
  s ← Σ_CT.Sign(sk_CT, (lid, t, c)) / SCT
  crt ← (c, s, Σ_CA.Sign(sk_CA, (c, s)))
  θ ← (u, Σ_TS.Sign(sk_TS, u)), u = (H(σ), t)
  b ← (m, σ, crt); Λ' ← Λ || H(0x00 || b)
  ck ← (org, |Λ'|, MTH(Λ'), t)
  return (Λ', α = (crt, θ, i, Path(Λ', i), ck, Σ_L.Sign(sk_L, ck)))
    
```

```

SS.Vf(ts, id, m, β)
  parse β = (crt, σ, θ, i, pf, ck, ζ_L), crt = (c, s, ζ_CA)
  if Σ_CT.Vf(pk_CT, (lid, t, c), s) = 0 then return 0
  if Σ_CA.Vf(pk_CA, (c, s), ζ_CA) = 0 then return 0
  if Σ_e.Vf(pk_e, m, σ) = 0 then return 0
  if Σ_TS.Vf(pk_TS, u, θ) = 0 then return 0
  if Σ_L.Vf(pk_L, ck, ζ_L) = 0 then return 0
  if PathVf(H(0x00 || (m, σ, crt)), i, pf, ck) = 0 then return 0
  return [(iss, sub) in c equals id ∧ t ∈ [t_c, t_c + Δ_c]]
    
```

Figure 2: Sigstore in the syntax of Definition 3.1, with the notation of Figure 1. Prep performs steps ③ and ⑨, Issue performs steps ⑤ to ⑧ and ⑪ to ⑬ and places the log body b at leaf i , and Vf performs step ⑯ with the inclusion proof pf .

The scheme is correct if an honest execution always yields $\text{Vf} = 1$ for the identity and message used.

The trust state ts carries the time dependence of the system. In Sigstore it holds the public keys and root certificates distributed through the update framework [26, 42], and in Section 4 it may also hold a log root, so that we can speak about a verifier that fixed its trust material at one time and checks a bundle at a later one.

3.2 The Sigstore instantiation

Figure 2 restates the flow of Figure 1 in the syntax of Definition 3.1, following the client, certificate authority and transparency service specifications [46, 47, 49]. It involves the schemes Σ_I of the identity provider, Σ_e of the ephemeral signer, Σ_{CA} of Fulcio, Σ_{CT} of the CT log, Σ_{TS} of the timestamp authority and Σ_L of the Rekor checkpoint, which Table 1 lists.

Two properties drive what follows. The identity binding and the artifact binding sit in different objects, so a verifier checks five signatures whose sizes add up. The log entry stores the certificate, with the ephemeral public key and two signatures, so log storage grows with the signature scheme as fast as the bundle. The public

Table 1: The authenticators of one Sigstore signing event, with the object each scheme signs, the algorithm deployed today and the bytes of the signature and of any public key that travels with it. The last row is the inclusion proof, which grows with the number N of log entries.

Scheme	Signed object	Deployed	Bytes
Σ_I	identity token	RSA-2048	256
Σ_e	artifact digest, possession	ECDSA P-256	91+142
Σ_{CA}	certificate body and SCT	ECDSA P-384	103
Σ_{CT}	certificate body and time	ECDSA P-256	71
Σ_{TS}	timestamp information	ECDSA P-256	71
Σ_L	log checkpoint	Ed25519	64
H	log inclusion proof	SHA-256	$32\lceil \log_2 N \rceil$

instance holds more than a hundred million such entries [47], so a migration that multiplies the size of a signature multiplies the storage of its operator by the same factor, as it does for the CT log that Fulcio writes to [29]. Under the hybrid model a verifier also accepts a signature under a certificate that expired years ago because a timestamp says the signature preceded the expiry, so whatever authenticates that timestamp stays in the trusted base for as long as the artifact is installed [2, 10]. The trust root itself reaches the verifier through the update framework under signatures of its own [26, 42], so a verifier that refreshes it after a key compromise inherits the same exposure, which is why Section 4 fixes the trust state at a known time.

4 Post-Quantum Security of Sigstore

4.1 Unforgeability

Definition 4.1 (Bundle unforgeability). For a transparent keyless signing scheme Π and an adversary $\mathcal{A}$, the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{uf}}(\lambda)$ runs $(pp, sk_S) \leftarrow \text{Setup}(1^\lambda)$, gives pp to $\mathcal{A}$, and provides four oracles, in which $\mathcal{O}_{\text{tok}}(id)$ returns a token for id and marks id corrupted; $\mathcal{O}_{\text{sig}}(id, m)$ runs an honest signing flow for id on m , returns the bundle and stores the pair (id, m) ; $\mathcal{O}_{\text{iss}}(\tau, \rho)$ submits an adversarial request to the services, where the certificate component acts only on a token that verifies while the transparency component admits any entry whose verification material is internally consistent, which is the deployed admission rule; $\mathcal{O}_{\text{view}}$ returns $\text{View}(\Lambda)$. The signing oracle does not hand out the token, which travels to the services over an authenticated channel. The adversary outputs (id^*, m^*, β^*) and wins if $\text{Vf}(ts, id^*, m^*, \beta^*) = 1$, the identity id^* was never marked corrupted, and (id^*, m^*) was never stored. We write $\text{Adv}_{\Pi}^{\text{uf}}(\mathcal{A})$ for the winning probability.

Sigstore unforgeability reduces to four assumptions.

THEOREM 4.2. *Let SS be Sigstore as in Figure 2, and let $\mathcal{A}$ be an adversary that makes q_s signing queries and q_i issuance queries. There exist adversaries $\mathcal{B}_1, \dots, \mathcal{B}_4$, each running in essentially the time of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{SS}}^{\text{uf}}(\mathcal{A}) \leq \text{Adv}_{\Sigma_I}^{\text{uf}}(\mathcal{B}_1) + \text{Adv}_{\Sigma_{CA}}^{\text{uf}}(\mathcal{B}_2) + q_s \text{Adv}_{\Sigma_e}^{\text{uf}}(\mathcal{B}_3) + \text{Adv}_H^{\text{cr}}(\mathcal{B}_4).$$

A forged bundle either carries a certificate the honest authority never signed, which is a forgery against Σ_{CA} , or one it signed against a token for an uncorrupted identity. That token is a forgery against Σ_I

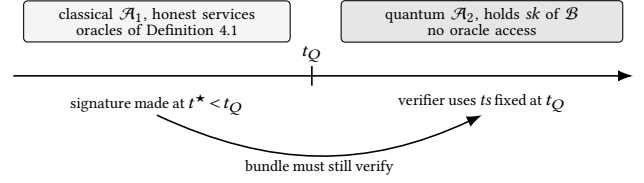


Figure 3: The timeline of Definition 4.3. Before the quantum epoch t_Q the classical $\mathcal{A}_1$ uses the oracles of Definition 4.1 against honest services. At t_Q the trust state ts is frozen and the quantum $\mathcal{A}_2$ receives the state of $\mathcal{A}_1$ and the secret keys of the break set $\mathcal{B}$, and it wins with a bundle accepted for an honest identity at a claimed signing time $t^* < t_Q$.

unless an honest signing query produced it, and then the certificate binds an honest ephemeral key and the artifact signature on a fresh message is a forgery against Σ_e ; the remaining case is a hash collision that makes a stored entry parse as a different one. The bound omits Σ_{CT} , Σ_{TS} and Σ_L , which carry auditability and time, so a compromise of one of them alone is detectable but gives no forgery. The term for Σ_e comes from the deployed admission rule. Rekord accepts any entry whose signature checks against the verification material it carries, so an adversary may pair a certificate from the public log with a signature of its own. The factor q_s is the loss from guessing which honest ephemeral key it targets, which makes a multi-instance notion the right one for the signer (Section 5), and Section 6 removes the term by binding identity and artifact in one admitted object.

4.2 Harvest now, forge later

Definition 4.1 is a snapshot and cannot express that a bundle produced in 2026 is checked in 2036 by a verifier whose trust state was fixed long before. We therefore split the experiment at a quantum epoch t_Q , in the sense in which the analysis of SSH by Benčina et al. splits its channel experiment to capture harvest-now-decrypt-later attacks [5], with the timeline of Figure 3.

Definition 4.3 (Harvest-now-forge-later unforgeability). Let $\mathcal{B}$ be a set of signature schemes, called the break set, and let t_Q be a time. The experiment $\text{Exp}_{\Pi, \mathcal{B}, \mathcal{A}}^{\text{hnfl}}(\lambda)$ runs the first phase exactly as Definition 4.1 with a classical adversary $\mathcal{A}_1$, until the log clock reaches t_Q . It then fixes the verifier trust state ts , hands $\mathcal{A}_1$'s state, the public view $\text{View}(\Lambda)$ and the secret keys of all schemes in $\mathcal{B}$ to a quantum adversary $\mathcal{A}_2$, and withdraws all oracles. The pair wins if $\mathcal{A}_2$ outputs (id^*, m^*, β^*) with $\text{Vf}(ts, id^*, m^*, \beta^*) = 1$, a claimed signing time earlier than t_Q , an identity never corrupted, and a pair (id^*, m^*) never signed. We write $\text{Adv}_{\Pi, \mathcal{B}}^{\text{hnfl}}(\mathcal{A})$ for the winning probability.

Changing signature schemes alone gives no such guarantee.

THEOREM 4.4. *Let SS be Sigstore with arbitrary signature schemes, and let the break set be $\mathcal{B} \supseteq \{\Sigma_{CA}, \Sigma_{CT}, \Sigma_{TS}, \Sigma_L\}$. There is an adversary $\mathcal{A}$ that makes no oracle query, runs in the time of one honest issuance, and achieves $\text{Adv}_{\text{SS}, \mathcal{B}}^{\text{hnfl}}(\mathcal{A}) = 1$ for every identity and every message.*

The reason is structural. Issuance, publication and time rest on four signatures under long lived public keys. The adversary generates a key pair, writes a certificate body for the victim identity with a validity window before t_Q , signs it with the recovered authority key,

produces the matching signed certificate timestamp and timestamp token, builds a tree containing its entry, and signs the checkpoint. All checks of step ⑩ in Figure 1 pass, and the token is never involved, because the adversary occupies the position that would check it. The theorem thus holds for a fully post-quantum deployment once the break set is reached, and transparency does not help, since the fabricated leaf appears only in a tree built for the one bundle handed to one verifier. What the verifier lacks is a commitment, fixed while the services were honest, to what the log contained.

A hash anchor fixed before t_Q restores security.

THEOREM 4.5. *Let Π be a transparent keyless signing scheme whose verifier trust state contains a root $\Lambda^\dagger = \text{MTH}(L^\dagger)$ of the log at time t_Q , and whose verification requires an inclusion proof of the bundle leaf against $\Lambda^\dagger$ and a leaf that binds the identity and the message. For every break set $\mathcal{B}$ and every adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ whose quantum phase makes q queries to the tree hash there are a classical adversary $\mathcal{D}$ in the sense of Definition 4.1 and a collision finder C with*

$$\text{Adv}_{\Pi, \mathcal{B}}^{\text{hfl}}(\mathcal{A}) \leq \text{Adv}_{\Pi}^{\text{uf}}(\mathcal{D}) + \text{Adv}_H^{\text{cf}}(C) \leq \text{Adv}_{\Pi}^{\text{uf}}(\mathcal{D}) + \frac{C(q+1)^3}{2^{8n_h}}$$

for an absolute constant C , where n_h is the output length of H in bytes.

The argument compares the inclusion path of the bundle leaf with the chain of nodes that $\text{MTH}(L^\dagger)$ computes at the same index. If the leaf and its preimage agree with those of $L^\dagger$, the entry was written before t_Q while the services were honest, so the classical phase alone has forged, which gives $\mathcal{D}$. Otherwise the two chains start apart and end at the same root $\Lambda^\dagger$, so at the first level where they meet two distinct inputs have the same image, which is the collision that C returns.

The two terms separate the two phases. Whatever the break set contains, the quantum phase contributes only a collision on the tree hash, since the anchor is a hash value the verifier already holds, and the signature assumptions appear once, against an adversary acting while the services held their keys. A deployment must therefore publish the log root widely at a known time and verify against it, and it loses the guarantee once it refreshes the anchor past t_Q , since a refreshed root commits to whatever a compromised service has appended; Section 7 exhibits both sides. Together, Theorems 4.4 and 4.5 show that Definition 4.1 does not imply Definition 4.3. A signature carries meaning as long as its key does, and only a value held before the keys were lost gives meaning that outlives them, which is why a time stamping service commits to a published value [12, 20] and why the anchor is a hash and not a stronger signature.

5 Adaptive One-Time Security in the QROM

We now resolve the primitive gap described in Section 1. Write $\text{WOTS-TW}[P, n, w]$ for the Winternitz variant of SLH-DSA with public parameter P , n byte chains and Winternitz parameter w , whose message encoding uses $\ell_1 = \lceil 8n/\log_2 w \rceil$ chains for the message and $\ell_2 = \lfloor \log_2(\ell_1(w-1)) / \log_2 w \rfloor + 1$ chains for the checksum, so that $\ell = \ell_1 + \ell_2$ [21, 23, 35]. The scheme signs n byte digests, and its public key is the tweakable hash of the ℓ chain endpoints.

Definition 5.1 (Randomized one-time signature). Let Th be a tweakable hash function and let H_m be a hash function with n byte output. The scheme $\text{ROTS}[\text{Th}, H_m, n, w]$ is defined as follows. Key generation samples a key identifier $\text{kid} \leftarrow \{0, 1\}^{128}$ and a seed, sets the address

$A = \text{kid} \parallel \text{tag}$, and outputs the WOTS-TW public key pk under (P, A) . Signing a message m samples $R \leftarrow \{0, 1\}^{8n}$, computes the digest $\mu = H_m(R \parallel pk \parallel m)$, and outputs (R, ς) with $\varsigma = \text{WOTS-TW}.\text{Sign}(\mu)$. Verification recomputes μ , recovers a candidate public key from ς and μ , and compares it with pk .

Two design points matter for the proof. The randomizer R is fresh at every signature, which is what makes the digest unpredictable to an adversary that already holds the public key, and pk enters the digest, which separates the target sets of different instances and removes the multi-target speedup [24]. The adaptive multi-instance bound is as follows.

THEOREM 5.2. *Let H_m be modeled as a quantum random oracle. For every quantum adversary $\mathcal{A}$ against the d instance one-time adaptive unforgeability of ROTs that makes q queries to H_m , there is an adversary $\mathcal{B}$ against the d instance non-adaptive unforgeability of WOTS-TW, running in essentially the time of $\mathcal{A}$, such that*

$$\text{Adv}_{\text{ROTS}}^{d\text{-euf1}}(\mathcal{A}) \leq \text{Adv}_{\text{WOTS-TW}}^{d\text{-na}}(\mathcal{B}) + \frac{3d}{2} \sqrt{\frac{q}{2^{8n}}} + \frac{8(q+1)^2}{2^{8n}}.$$

The non-adaptive challenger asks for the signed digests before it produces the keys, so $\mathcal{B}$ draws d uniform digests, receives the public keys and the matching signatures, and runs $\mathcal{A}$ on them. On an adaptive signing query, $\mathcal{B}$ picks a fresh randomizer, reprograms H_m at that point to the digest it holds and returns the stored signature, which costs the second term by the bound of Grilo et al. A forgery then either uses a new digest, which is a non-adaptive forgery, or the signed digest on a new input, which is a preimage of a uniform value in a domain separated by the public key and costs the third term. Combined with the tight non-adaptive bound of Hülsing and Kudinov [23], the theorem yields a bound in terms of the tweakable hash alone. That bound reads as follows.

COROLLARY 5.3. *With $\mathcal{B}$ as in Theorem 5.2 and a pseudorandom function F used to expand the chain seeds,*

$$\begin{aligned} \text{Adv}_{\text{ROTS}}^{d\text{-euf1}}(\mathcal{A}) \leq & \text{Adv}_F^{\text{prf}}(\mathcal{B}_1) + \text{Adv}_{\text{Th}, d, w}^{\text{tcr}}(\mathcal{B}_2) + \text{Adv}_{\text{Th}, d, \ell}^{\text{pre}}(\mathcal{B}_3) \\ & + w \text{Adv}_{\text{Th}, d, \ell}^{\text{ud}}(\mathcal{B}_4) + \frac{3d}{2} \sqrt{\frac{q}{2^{8n}}} + \frac{8(q+1)^2}{2^{8n}}. \end{aligned}$$

At the first NIST category we take $n = 16$ and $w = 16$, so $\ell_1 = 32$, $\ell_2 = 3$ and $\ell = 35$, the parameters of the 128 bit SHA-2 sets of SLH-DSA [35], and at the third and fifth categories $n = 24$ and $n = 32$. For $d = 2^{40}$ instances, which matches the scale of the public Rekord log, Table 2 gives the number of quantum queries at which each additive term of Theorem 5.2 reaches one. The digest term sits at the Grover bound for an $8n$ bit preimage, which SLH-DSA carries at the same n . The reprogramming term depends on the length of R alone, because the proof uses that length nowhere else, and with $|R| = 8n$ it is the binding term at the first category. Drawing R of 32 bytes moves it past the digest term at a cost of 16 bytes per bundle at the first category and 8 at the third; our measurements use $|R| = 8n$.

A 16 byte chain would be too short for a plain hash at this scale, since a search over d targets costs only $\sqrt{2^{8n}/d}$ quantum operations, and the distinct address of each instance forbids that search as in SLH-DSA [24]. Addresses carry a 128 bit key identifier, two honest ones collide with probability at most $d^2 2^{-128}$, and the service refuses a repeated identifier. The tree over entries keeps the full 32 byte

Table 2: Parameters of ROTS and the base two logarithm of the number of quantum queries at which each additive term of Theorem 5.2 reaches one, for $d = 2^{40}$ instances. The columns give the NIST category, the chain length n in bytes, the number ℓ of chains, the one-time signature size in bytes, the reprogramming term for a randomizer of $8n$ and of 256 bits, and the digest term.

Cat.	n	ℓ	$ \zeta_{\text{ots}} $	$ R =8n$	$ R =256$	Digest
1	16	35	560	46.8	174.8	62.5
3	24	51	1224	110.8	174.8	94.5
5	32	67	2144	174.8	174.8	126.5

output of RFC 6962, because there the adversary chooses both sides of a collision.

6 The Latch Construction

6.1 Design

Sections 4 and 5 suggest two changes. The signer should use a one-time hash-based key, which is smaller and faster than a lattice key and now has an adaptive proof, and the meaning of a bundle should be carried by a hash anchor instead of a stack of signatures. Latch does both and fuses the four service assertions of Table 1 into one.

The construction runs in epochs of duration Δ , the batching interval the transparency service already uses [47]. In each epoch the anchor service collects up to B requests, forms one leaf per request, builds one tree and signs one head. The head carries the epoch time and replaces the timestamp token, the leaf carries the identity and the one-time public key and replaces the certificate, the published tree replaces the signed certificate timestamp, and the head is the checkpoint. Figures 4 and 5 give the structure and the algorithms.

A bundle is completed in one of two modes. In self-contained mode it carries the epoch head with the signature of the anchor service. In anchored mode it carries an inclusion path of the head in the tree over all heads, checked against a root $\Lambda^\dagger$ the verifier already holds; since the verifier picks that root when it refreshes its trust state, the path is attached at distribution time and the log serves it in $O(\log E)$ from nodes it keeps. Anchored mode is what Theorem 4.5 asks for and is the smaller mode, since $\lceil \log_2 E \rceil$ hashes are shorter than a post-quantum signature, and its verifier holds one hash value instead of the list of batch roots of Merkle Tree Certificates [6].

6.2 Security

Latch instantiates Definition 3.1 and inherits the two theorems of Section 4. Its classical unforgeability is as follows.

THEOREM 6.1. *Let $\mathcal{A}$ make q_s signing queries. There exist $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ running in essentially the time of $\mathcal{A}$ with*

$$\text{Adv}_{\text{Latch}}^{\text{uf}}(\mathcal{A}) \leq \text{Adv}_{\Sigma_I}^{\text{euf}}(\mathcal{B}_1) + \text{Adv}_{\Sigma_A}^{\text{euf}}(\mathcal{B}_2) + \text{Adv}_H^{\text{cr}}(\mathcal{B}_3).$$

Compared with Theorem 4.2, the signer scheme and the factor q_s leave the bound and the service assumptions drop from three to two, because a Latch entry binds identity and artifact in one object that the service admits only against a token. The one-time signature still

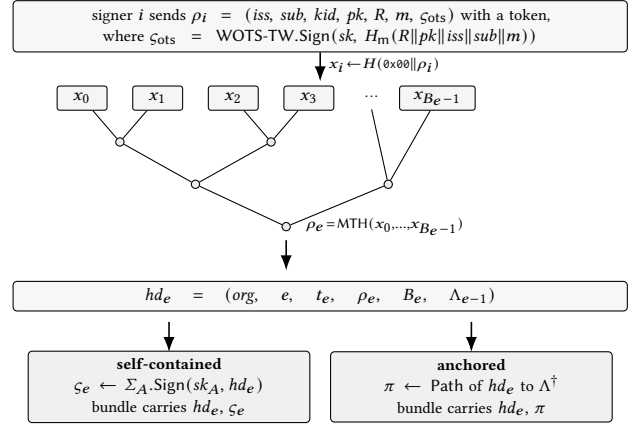


Figure 4: One Latch epoch. A request ρ_i holds the identity (iss, sub), a key identifier kid , a one-time public key pk , a randomizer R , the artifact digest m and a one-time signature ζ_{ots} over a digest that binds all of them. Admitted requests become the leaves x_i of the epoch tree with root ρ_e over $B_e \leq B$ leaves, and the head hd_e adds the log origin org , the epoch index e , the epoch time t_e and the root Λ_{e-1} of the tree over earlier heads. The head is completed by a signature ζ_e under the anchor key or by a path to a root $\Lambda^\dagger$ the verifier holds.

matters. It is the possession proof a certificate authority owes its subject [46], and it makes an entry attributable to the party that produced it, which is what a monitor acts on. The next theorem states both.

THEOREM 6.2. *Suppose $\forall f$ accepts β for (id, m) under head hd_e . Then, except with probability $\text{Adv}_H^{\text{cr}}(\mathcal{B}_1)$, the leaf that binds id, pk and m is committed by ρ_e inside hd_e ; and except with the further probability $\text{Adv}_{\text{ROTS}}^{q_s\text{-euf}1}(\mathcal{B}_2)$, its one-time signature was produced by the holder of pk . Consequently a monitor that reads all entries under the published heads observes each accepted signing event together with an attribution of the key that produced it, and a verifier that additionally checks the head against a witnessed view of Λ detects any equivocation by the service.*

The first part is the chain argument inside the epoch tree, and the second reduces to Theorem 5.2 by running the q_s honest one-time instances through the challenger of the multi-instance game, since a key the adversary chose itself is a key whose secret it holds.

Finally, the anchored mode instantiates Theorem 4.5 directly.

COROLLARY 6.3. *In anchored mode, in which the bundle carries the inclusion path of hd_e in the tree of heads and the verifier holds its root $\Lambda^\dagger$ fixed at t_Q , for every break set $\mathcal{B}$ there are a classical $\mathcal{D}$ and a collision finder $\mathcal{C}$ with*

$$\text{Adv}_{\text{Latch-A}, \mathcal{B}}^{\text{hnfl}}(\mathcal{A}) \leq \text{Adv}_{\Sigma_I}^{\text{euf}}(\mathcal{D}) + \text{Adv}_H^{\text{cr}}(\mathcal{C}).$$

The inclusion proofs in the epoch tree and in the tree of heads compose into one proof of the entry against $\Lambda^\dagger$, so Theorem 4.5 applies, and its classical term reduces to Σ_I because the service writes an entry for an identity only against a verifying token. Anchored verification checks no signature, so the quantum phase faces only collision resistance of SHA-256, and the anchor scheme leaves the statement.

```

Latch.Prepare(pp, id, m)
kid ← $ {0,1}^{128}; sd ← $ {0,1}^{8n}; A ← kid || tag
pk ← WOTS-TW.Kg(P, A, sd); R ← $ {0,1}^{8n}
μ ← Hm(R || pk || iss || sub || m)
ζots ← WOTS-TW.Sign(P, A, sd, μ)
return ρ = (iss, sub, kid, pk, R, m, ζots)

Latch.Issue(skA, Λ, {(τj, ρj)} j < Be, te)
for j = 0 to Be - 1 do
  if ΣI.Vf(pkj, τj) = 0 ∨ kidj ∈ K then return ⊥
  μj ← Hm(Rj || pkj || issj || subj || mj)
  if WOTS-TW.Vf(P, Aj, pkj, μj, ζots, j) = 0 then return ⊥
  K ← K ∪ {kidj}; xj ← H(0x00 || ρj)
ρe ← MTH(x0, ..., xBe-1)
hde ← (org, e, te, ρe, Be, Λe-1); ζe ← ΣA.Sign(skA, hde)
Λe ← MTH(Λe-1 leaves || H(0x00 || hde))
return {(j, Path(x0, j), hde, ζe)} j < Be

Latch.Vf(ts, id, m, β)
parse β = (ρ, j, pf, hde, ζe)
μ ← Hm(R || pk || iss || sub || m)
if WOTS-TW.Vf(P, kid || tag, pk, μ, ζots) = 0 then return 0
if PathVf(H(0x00 || ρ), j, Be, pf, ρe) = 0 then return 0
if ΣA.Vf(pkA, hde, ζe) = 0 then return 0 / or path to Λ†
return [(iss, sub) = id ∧ digest in ρ equals m]

```

Figure 5: The Latch algorithms, with the seed sd , the public parameter P , the address A and the set $\mathcal{K}$ of used key identifiers. One signature ζ_{ots} over the digest μ serves both as possession proof and as artifact signature. Issuance checks the token, rejects a used identifier, verifies ζ_{ots} , builds the epoch tree, signs the head once and appends it to the tree of heads with root Λ_e . Verification recomputes μ and checks ζ_{ots} , the path to ρ_e , the head and the policy.

6.3 Deployment and migration

The service enforces single use of a one-time key by refusing a repeated key identifier, a membership test on the index it already keeps for deduplication [47]. The epoch key signs once per epoch at a rate fixed in advance, so a stateful hash-based scheme becomes usable at the anchor and the objection of the project policy to such schemes at a certificate authority or a log no longer applies [15, 45]. The signer waits at most one epoch, the batching delay Rekord already imposes. Fusing issuance and logging concentrates trust, and independent witnesses offset this by cosigning each head, so that equivocation is detected as Theorem 6.2 states [13, 38]. Migration can be gradual, since a Latch leaf sits where the current entry type sits [47, 48]. By Theorem 4.4 old artifacts gain nothing from new algorithms, so the first step for them is to pin and publish the log root, which brings them under Theorem 4.5, while new bundles move to anchored mode later by attaching a path without the signer.

6.4 Complexity

Table 3 compares the work and space of one signing event, where n , w and ℓ are the one-time parameters, n_h is the tree hash length in bytes, B the epoch batch, N the number of log entries and E the number of epochs under the anchor. A one-time key generation costs ℓw hash calls, and signing and public key recovery each cost about $\ell(w-1)/2$ calls, since the exponents of the ℓ chains sum to about $\ell(w-1)/2$ on a uniform digest.

The formulas show three differences. The Latch bundle holds one service signature instead of five and none when anchored, so the dominant term falls from $5|\sigma|$ to $|\sigma|$ and then to zero. It depends on $\lceil \log_2 B \rceil$ instead of $\lceil \log_2 N \rceil$, so it does not grow as the log fills. The service performs a constant number of hash calls per event and amortizes one signature over B events instead of signing three times per event. In exchange Latch pays $\Theta(\ell w)$ hash calls, about 560 chain steps at $\ell=35$, $w=16$ and $n=16$, which is far cheaper than the five lattice verifications it replaces.

7 Evaluation

7.1 Setup

We implemented Latch and six alternatives on a common substrate. The hash-based core, the tweakable hash function, the RFC 6962 trees, LM-OTS and LMS are written in C against the SHA-256 of OpenSSL; ML-DSA, FN-DSA, SLH-DSA and XMSS come from liboqs [40], and ECDSA and Ed25519 from OpenSSL. All methods emit the same length-delimited wire format, so the sizes differ only in the cryptographic objects carried, as in the deployed bundle format [49]. We measure on an Apple M-series workstation, single threaded, with medians over five to forty repetitions, $N = 2^{27}$ and $B = 1024$. Anchored variants use a tree of $E = 2^{20}$ heads with the measured epoch at its midpoint, and proofs against the full log materialize only the co-path, which gives the exact verifier cost. The alternatives are the direct substitutions with ML-DSA [17, 34], FN-DSA [36] and SLH-DSA [7, 35]; XMSS [11, 22] or LMS [30] at the services with ML-DSA at the signer, the split that statefulness forces [15, 45]; and per-service batching in the style of Merkle Tree Certificates [6], self-contained and anchored. The classical deployment serves as a reference.

7.2 Cost of one signing event

In Table 4 Latch is the smallest and the fastest in all columns of both categories. Against the direct ML-DSA substitution, the migration the project experiments with, Latch cuts the bundle by 4.1×, log storage by 12.4×, signer time by 3.8×, service time by 16.4× and verifier time by 3.3×, and in anchored mode the bundle falls by 8.2× and verification by 6.2×. Against per-service batching, the strongest alternative, the reductions are 3.5× in size and 2.6× in verification time, and 4.0× and 2.3× when both are anchored. An anchored Latch bundle of 1795 bytes is within seven percent of the 1687 bytes of the deployed elliptic curve pipeline and verifies 11.6× faster, so a post-quantum transition need not cost bandwidth.

Figure 6 plots the first category.

7.3 Scaling

The service cost of Latch flattens at about 0.025 milliseconds per event from $B=64$ on, as the epoch signature amortizes away, while

Table 3: Cost of one signing event, with sizes in bytes and times in operations. $|\sigma|$ and $|pk|$ are the signature and public key sizes of the service scheme, T_g, T_s, T_v its key generation, signing and verification costs, T_h one tweakable hash call and c the common wire format overhead. In anchored rows the verifier holds a root over the epoch heads, so a path of $\lceil \log_2 E \rceil$ nodes replaces the epoch signature. The last column is the log storage per entry.

Method	Bundle	Signer	Service	Verifier	Log entry
Sigstore	$ pk +5 \sigma +n_h \lceil \log_2 N \rceil + c$	T_g+2T_s	T_v+4T_s	$5T_v+(\lceil \log_2 N \rceil+2)T_h$	$ pk +3 \sigma +c$
Batched (MTC)	$ pk +4 \sigma +3n_h \lceil \log_2 B \rceil + c$	T_g+2T_s	T_v+3T_s/B	$4T_v+(3\lceil \log_2 B \rceil+6)T_h$	$ pk + \sigma +c$
Batched, anchored	$ pk + \sigma +3n_h(\lceil \log_2 B \rceil+\lceil \log_2 E \rceil)+c$	T_g+2T_s	T_v+3T_s/B	$T_v+(3\lceil \log_2 B \rceil+3\lceil \log_2 E \rceil+6)T_h$	$ pk + \sigma +c$
Latch	$\ell n+2n+n_h \lceil \log_2 B \rceil + \sigma + c$	$(\ell w + \frac{\ell(w-1)}{2})T_h$	$\frac{\ell(w-1)}{2}T_h + \frac{T_s}{B}$	$T_v + (\frac{\ell(w-1)}{2} + \lceil \log_2 B \rceil)T_h$	$\ell n+2n+c$
Latch-A	$\ell n+2n+n_h(\lceil \log_2 B \rceil+\lceil \log_2 E \rceil)+c$	$(\ell w + \frac{\ell(w-1)}{2})T_h$	$\frac{\ell(w-1)}{2}T_h + \frac{T_s}{B}$	$(\frac{\ell(w-1)}{2} + \lceil \log_2 B \rceil + \lceil \log_2 E \rceil)T_h$	$\ell n+2n+c$

Table 4: Cost of one signing event with $B = 1024$ and $N = 2^{27}$. Bundle and log are in bytes, and signer, service and verifier in milliseconds per event, with the service including its share of the epoch signature. In the first and third NIST category blocks the last two rows are ours, and the classical rows are the deployed pipeline as a reference.

Method	Bundle	Log	Signer	Service	Verifier
<i>First NIST category</i>					
ML-DSA-44	14659	8848	0.172	0.409	0.173
FN-DSA-512	5420	3144	4.538	0.701	0.139
SLH-DSA-128f	86734	51584	38.85	77.40	5.907
SLH-DSA-128s	40566	23883	845.8	1587.0	2.024
XMSS services	14979	9008	0.171	6.653	1.231
LMS services	10755	6896	0.172	0.715	0.712
Batched (MTC)	12472	3958	0.177	0.055	0.135
Batched, anchored	7162	3958	0.178	0.056	0.064
Latch	3565	712	0.045	0.025	0.052
Latch-A	1795	712	0.045	0.025	0.028
<i>Third NIST category</i>					
ML-DSA-65	19744	12155	0.273	0.589	0.269
FN-DSA-1024	9392	5884	14.522	1.319	0.259
SLH-DSA-192f	179630	107328	63.60	126.9	8.543
SLH-DSA-192s	82423	49004	1480.1	2823.8	2.968
Batched (MTC)	16668	5487	0.271	0.078	0.212
Batched, anchored	8691	5487	0.275	0.078	0.086
Latch	5134	1392	0.099	0.039	0.088
Latch-A	2475	1392	0.087	0.039	0.043
<i>Classical reference</i>					
ECDSA P-256	1687	575	0.107	0.265	0.324
Ed25519	1602	505	0.189	0.428	0.446

the direct substitution stays near 0.42 milliseconds at any batch size, since its three signatures per event cannot be shared. The Latch bundle grows by 32 bytes per doubling of B , from 3244 bytes at $B = 1$ to 3693 bytes at $B = 2^{14}$, and not at all with N , whereas the direct substitution grows by 32 bytes per doubling of N . Of the 14659 bytes of the direct substitution, 12100 are five ML-DSA signatures, 1312 the ephemeral public key and 864 the inclusion proof, while of the 3565 bytes of Latch, 2420 are the epoch signature, 560 the one-time signature and 320 the epoch path, and Latch-A carries no service signature at all.

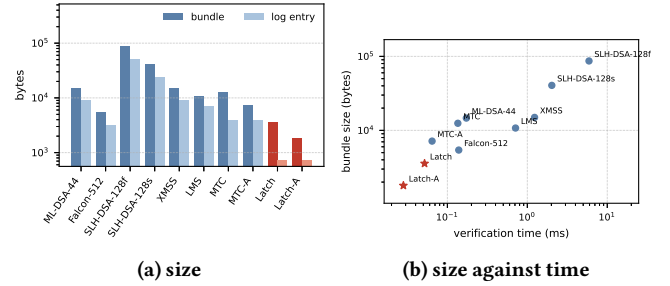


Figure 6: Cost of one signing event at the first NIST category. In (a) the darker bar of each pair is the bundle and the lighter bar the log entry, in bytes on a logarithmic scale, with our two pairs rightmost. In (b) each method is placed by verification time in milliseconds and bundle size in bytes on logarithmic axes, with our two modes as stars, and no other method lies below or to the left of either of them.

At $n = 16$ and $B = 64$ the Winternitz choices $w \in \{4, 16, 256\}$ give one-time signatures of 1088, 560 and 288 bytes, bundles of 3965, 3437 and 3165 bytes, and verification in 0.048, 0.053 and 0.138 milliseconds. We keep $w = 16$, the value of SLH-DSA, since $w = 4$ buys a 9 percent faster verification for a 15 percent larger bundle and $w = 256$ saves 8 percent of size for a 2.6 $\times$ slower verification, in a system that verifies far more often than it signs.

7.4 Harvest now, forge later

Table 5 runs the experiment behind Theorems 4.4 and 4.5. We build an honest history, freeze the trust state at t_Q , give the adversary the service secret keys, and ask for a bundle for an identity it never controlled, dated one day before t_Q . All constructions rooted in a signature accept the forgery, including self-contained Latch, at the cost of one honest issuance, and both anchored constructions reject it. A verifier that refreshes its anchor after t_Q accepts the forgery again, so the value of an anchor lies in when it was pinned.

7.5 Throughput and model validation

On one core the service column inverts to 40 thousand events per second for Latch, 18 thousand for batching, 2.4 thousand for the direct substitution, 150 for stateful services and less than one for the small stateless set, against a few entries per second on the public instance, so the difference is headroom for release peaks. Witness

Table 5: The harvest-now-forge-later experiment. The adversary holds all service secret keys, as after t_Q , and forges a bundle for an identity it never controlled, dated one day before t_Q , and the columns give whether a verifier accepts it and the adversary time. Rows one to three confirm Theorem 4.4, rows four and six confirm Theorem 4.5, and row five shows that the anchor must be pinned before t_Q .

Construction and anchor	Accepted	Cost (ms)
ML-DSA-44, signature anchored	yes	0.68
Batched (MTC), signature anchored	yes	0.44
Latch, signature anchored	yes	0.14
Latch-A, root pinned at t_Q	no	3.12
Latch-A, root refreshed after t_Q	yes	3.12
Batched, root pinned at t_Q	no	9.13

cosignatures separate the two Latch modes, since two ML-DSA-44 cosignatures add 4840 bytes to a self-contained bundle and nothing to an anchored one, whose witnesses attest the root the verifier already holds once per trust refresh.

A Latch verification at the first category performs 276.2 hash calls on average and one lattice verification, against the 275.5 that Section 6.4 predicts; a signer performs 855.5 against 842.5 and the service 270.5 against 265.5. Predicted bundle sizes are within five percent of the measured ones, at 3580 against 3565, 1800 against 1795 when anchored, 14508 against 14659 for the direct substitution and 6812 against 7162 for anchored batching. When several artifacts from one epoch are verified, as for a container image, the head travels once, so each further artifact costs 1020 bytes against 5032 and 14659, and sixteen cost 18.5 kibibytes against 85.9 and 229.

8 Related Work

Newman et al. presented Sigstore with an informal attacker model and adoption measurements [37], and Merrill et al. replaced its clear-text identities with commitments [33]; neither gives a security definition or considers quantum adversaries. The update framework protects repositories from key compromise [26, 42], in-toto binds build steps into a verifiable chain [50], and CHAINIAC combines collective signing with verified builds [38]. Our model builds on Certificate Transparency [28, 29] and key transparency [31], secure logging has a game based treatment for the log alone [16], the hybrid validation model is due to Braun et al. [10], and fixing a document in time by a published commitment goes back to Haber and Stornetta [12, 20]. Protocol level post-quantum analyses have focused on channel establishment, from KEMTLS [43] to SSH [5]. Merkle Tree Certificates [6] batch issuance into a tree to shorten post-quantum certificate chains and form our closest baseline, but they batch one service, assume an up to date relying party, and leave the ephemeral key and the transparency log aside. The hash-based schemes we use run from Merkle and Winternitz to the current standards [7, 21, 32, 35], and the proofs of their central component were repaired, tightened and mechanized in works we build on [3, 4, 23].

Future Directions

A natural next step is a machine-checked version of Theorem 5.2, which would place the adaptive quantum argument on the same footing as the classical proofs that have already been mechanized for XMSS and SLH-DSA. A second direction is an operational study of how log roots should be pinned and distributed so that the anchoring guarantee of Theorem 4.5 covers artifacts signed long before a migration begins.

References

- [1] Josh Aas, Richard Barnes, Benton Case, Zakir Durumeric, Peter Eckersley, Alan Flores-López, J. Alex Halderman, Jacob Hoffman-Andrews, James Kasten, Eric Rescorla, Seth Schoen, and Brad Warren. 2019. Let’s Encrypt: An Automated Certificate Authority to Encrypt the Entire Web. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2473–2487.
- [2] Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. 2001. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). RFC 3161, IETF.
- [3] Manuel Barbosa, François Dupressoir, Benjamin Grégoire, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. 2023. Machine-Checked Security for XMSS as in RFC 8391 and SPHINCS+. In *Advances in Cryptology – CRYPTO 2023 (LNCS, Vol. 14085)*. Springer, 421–454.
- [4] Manuel Barbosa, François Dupressoir, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. 2024. A Tight Security Proof for SPHINCS+, Formally Verified. In *Advances in Cryptology – ASIACRYPT 2024 (LNCS, Vol. 15488)*. Springer, 33–65.
- [5] Benjamin Benčina, Benjamin Dowling, Varun Maram, and Keita Xagawa. 2025. Post-quantum Cryptographic Analysis of SSH. In *2025 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 595–613.
- [6] David Benjamin, Devon O’Brien, Bas Westerbaan, Luke Valenta, and Filippo Valsorda. 2026. Merkle Tree Certificates. Internet-Draft draft-ietf-plants-merkle-tree-certs, IETF.
- [7] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. 2019. The SPHINCS+ Signature Framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2129–2146.
- [8] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. 2011. Random Oracles in a Quantum World. In *Advances in Cryptology – ASIACRYPT 2011 (LNCS, Vol. 7073)*. Springer, 41–69.
- [9] Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. Quantum Cryptanalysis of Hash and Claw-Free Functions. In *LATIN ’98: Theoretical Informatics (LNCS, Vol. 1380)*. Springer, 163–169.
- [10] Johannes Braun, Franziskus Kiefer, and Andreas Hülsing. 2013. Revocation and Non-repudiation: When the First Destroys the Latter. In *Public Key Infrastructures, Services and Applications (EuroPKI) (LNCS, Vol. 8341)*. Springer, 31–46.
- [11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. 2011. XMSS – A Practical Forward Secure Signature Scheme Based on Minimal Security Assumptions. In *Post-Quantum Cryptography (PQCrypto) (LNCS, Vol. 7071)*. Springer, 117–129.
- [12] Ahto Buldas and Sven Laur. 2007. Knowledge-Binding Commitments with Applications in Time-Stamping. In *Public Key Cryptography – PKC 2007 (LNCS, Vol. 4450)*. Springer, 150–165.
- [13] C2SP Authors. 2025. Transparency Log Witness Protocol. <https://github.com/C2SP/C2SP/blob/main/tlog-witness.md>.
- [14] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, and Tim Polk. 2008. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280, IETF.
- [15] David A. Cooper, Daniel C. Apon, Quynh H. Dang, Michael S. Davidson, Morris J. Dworkin, and Carl A. Miller. 2020. *Recommendation for Stateful Hash-Based Signature Schemes*. Technical Report NIST Special Publication 800-208. National Institute of Standards and Technology.
- [16] Benjamin Dowling, Felix Günther, Udyani Herath, and Douglas Stebila. 2016. Secure Logging Schemes and Certificate Transparency. In *Computer Security – ESORICS 2016 (LNCS, Vol. 9879)*. Springer, 140–158.
- [17] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2018, 1 (2018), 238–268.
- [18] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. 1988. A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks. *SIAM J. Comput.* 17, 2 (1988), 281–308.
- [19] Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. 2021. Tight Adaptive Reprogramming in the QROM. In *Advances in Cryptology – ASIACRYPT 2021 (LNCS, Vol. 13090)*. Springer, 637–667.
- [20] Stuart Haber and W. Scott Stornetta. 1990. How to Time-Stamp a Digital Document. In *Advances in Cryptology – CRYPTO ’90 (LNCS, Vol. 537)*. Springer, 437–455.

- [21] Andreas Hülsing. 2013. W-OTS+ – Shorter Signatures for Hash-Based Signature Schemes. In *Progress in Cryptology – AFRICACRYPT 2013 (LNCS, Vol. 7918)*. Springer, 173–188.
- [22] Andreas Hülsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. 2018. XMSS: eXtended Merkle Signature Scheme. RFC 8391, IETF.
- [23] Andreas Hülsing and Mikhail Kudinov. 2022. Recovering the Tight Security Proof of SPHINCS+. In *Advances in Cryptology – ASIACRYPT 2022 (LNCS, Vol. 13794)*. Springer, 3–33.
- [24] Andreas Hülsing, Joost Rijneveld, and Fang Song. 2016. Mitigating Multi-target Attacks in Hash-Based Signatures. In *Public-Key Cryptography – PKC 2016 (LNCS, Vol. 9614)*. Springer, 387–416.
- [25] Don Johnson, Alfred Menezes, and Scott Vanstone. 2001. The Elliptic Curve Digital Signature Algorithm (ECDSA). *International Journal of Information Security* 1, 1 (2001), 36–63.
- [26] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. 2016. Diplomat: Using Delegations to Protect Community Repositories. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 567–581.
- [27] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 1509–1526.
- [28] Ben Laurie, Adam Langley, and Emilia Kasper. 2013. Certificate Transparency. RFC 6962, IETF.
- [29] Ben Laurie, Eran Messeri, and Rob Stradling. 2021. Certificate Transparency Version 2.0. RFC 9162, IETF.
- [30] David McGrew, Michael Curcio, and Scott Fluhrer. 2019. Leighton-Micali Hash-Based Signatures. RFC 8554, IETF.
- [31] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. 2015. CONIKS: Bringing Key Transparency to End Users. In *24th USENIX Security Symposium*. USENIX Association, 383–398.
- [32] Ralph C. Merkle. 1989. A Certified Digital Signature. In *Advances in Cryptology – CRYPTO ’89 (LNCS, Vol. 435)*. Springer, 218–238.
- [33] Kelsey Merrill, Zachary Newman, Santiago Torres-Arias, and Karen R. Sollins. 2023. Speranza: Usable, Privacy-friendly Software Signing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 3388–3402.
- [34] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Digital Signature Standard*. Technical Report FIPS 204. National Institute of Standards and Technology.
- [35] National Institute of Standards and Technology. 2024. *Stateless Hash-Based Digital Signature Standard*. Technical Report FIPS 205. National Institute of Standards and Technology.
- [36] National Institute of Standards and Technology. 2025. *FN-DSA: Fast-Fourier Transform over NTRU-Lattice-Based Digital Signature Standard*. Technical Report Draft FIPS 206. National Institute of Standards and Technology.
- [37] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2353–2367.
- [38] Kirill Nikitin, Eleftherios Kokoris-Kogias, Philipp Jovanovic, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Justin Cappos, and Bryan Ford. 2017. CHAINIAC: Proactive Software-Update Transparency via Collectively Signed Skipchains and Verified Builds. In *26th USENIX Security Symposium*. USENIX Association, 1271–1287.
- [39] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA) (LNCS, Vol. 12223)*. Springer, 23–43.
- [40] Open Quantum Safe Project. 2025. liboqs: C Library for Quantum-Safe Cryptography. <https://github.com/open-quantum-safe/liboqs>.
- [41] Nat Sakimura, John Bradley, Michael B. Jones, Breno de Medeiros, and Chuck Mortimore. 2014. OpenID Connect Core 1.0. OpenID Foundation Specification.
- [42] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. 2010. Survivable Key Compromise in Software Update Systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*. ACM, 61–72.
- [43] Peter Schwabe, Douglas Stebila, and Thom Wiggers. 2020. Post-Quantum TLS Without Handshake Signatures. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1461–1480.
- [44] Peter W. Shor. 1997. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM J. Comput.* 26, 5 (1997), 1484–1509.
- [45] Sigstore Authors. 2025. Cryptographic Algorithms and KMS Policy. <https://github.com/sigstore/sigstore/blob/main/POLICY.md>.
- [46] Sigstore Authors. 2025. Fulcio, A Certificate Authority for Code Signing. <https://github.com/sigstore/architecture-docs/blob/main/fulcio-spec.md>.
- [47] Sigstore Authors. 2025. Rekor v2, a Tiles-based Transparency Service. <https://github.com/sigstore/architecture-docs/blob/main/rekor-v2-spec.md>.
- [48] Sigstore Authors. 2025. Sigstore Algorithm Registry. <https://github.com/sigstore/architecture-docs/blob/main/algorithm-registry.md>.
- [49] Sigstore Authors. 2025. Sigstore Client Specification. <https://github.com/sigstore/architecture-docs/blob/main/client-spec.md>.
- [50] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*. USENIX Association, 1393–1410.
- [51] Mark Zhandry. 2019. How to Record Quantum Queries, and Applications to Quantum Indifferentiability. In *Advances in Cryptology – CRYPTO 2019 (LNCS, Vol. 11693)*. Springer, 239–268.

A Appendix

This appendix gives the complete proofs of the statements of the main body, together with the concrete parameters, the cost accounting rule, and the full experimental data. Throughout, q_s is the number of signing queries an adversary makes, q_i is the number of issuance queries, and q is the number of quantum queries to a public hash function; $\Pr[G_k]$ denotes the probability that the adversary wins in game G_k , and all reductions run their adversary once and otherwise perform the work of the honest parties, so their running time is that of the adversary plus $O(q_s + q_i)$ evaluations of the scheme algorithms.

A.1 Proof of Theorem 4.2

Let $\mathcal{A}$ be an adversary in $\text{Exp}_{\text{SS}, \mathcal{A}}^{\text{uf}}$ and write its output as (id^*, m^*, β^*) with $\beta^* = (crt^*, \sigma^*, \theta^*, i^*, pf^*, ck^*, \varsigma_L^*)$ and $crt^* = (c^*, s^*, \varsigma_{CA}^*)$.

Game G_0 is the experiment itself, so that $\Pr[G_0] = \text{Adv}_{\text{SS}}^{\text{uf}}(\mathcal{A})$.

Game G_1 is G_0 with one additional abort. The challenger keeps a list of all pairs $(x, H(x))$ that any honest party evaluates during the game, together with those evaluated while checking the forgery, and aborts as soon as two distinct inputs in that list share an output. An algorithm $\mathcal{B}_4$ that runs the whole of G_1 internally and outputs the two colliding inputs succeeds whenever the abort fires, so $|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}_H^{\text{cr}}(\mathcal{B}_4)$.

Game G_2 is G_1 with an abort if the pair (c^*, s^*) was never an argument of $\Sigma_{CA}.\text{Sign}$ inside an execution of Issue . We build $\mathcal{B}_2$ against the existential unforgeability of Σ_{CA} . It receives a challenge public key, places it as pk_{CA} in the public parameters, generates all other keys itself, and simulates the four oracles exactly as the challenger of G_1 would, except that each certificate signature is obtained by a query to its own signing oracle. When the abort event fires it outputs $((c^*, s^*), \varsigma_{CA}^*)$. The message was never queried, by the definition of the abort, and the signature verifies, because the winning condition includes the certificate check of Figure 2. Hence $|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{\Sigma_{CA}}^{\text{uf}}(\mathcal{B}_2)$.

In G_2 the pair (c^*, s^*) was produced inside an execution of Issue , and by the winning condition the identity encoded in c^* equals id^* . That execution first verified an identity token τ under pk_I and then copied the identity claim of τ into the certificate body, so the identity claim of τ is id^* .

Game G_3 is G_2 with an abort if that token was never an output of Auth during the game. We build $\mathcal{B}_1$ against the existential unforgeability of Σ_I . It places its challenge key as pk_I , answers every invocation of Auth , whether inside O_{tok} or inside O_{sig} , by a query to its signing oracle, and outputs τ when the abort fires. Hence $|\Pr[G_2] - \Pr[G_3]| \leq \text{Adv}_{\Sigma_I}^{\text{uf}}(\mathcal{B}_1)$.

In G_3 the token was produced by Auth for the identity id^* . Because id^* is never marked corrupted, the producing call was not O_{tok} , so it was a call $O_{\text{sig}}(id^*, m_k)$ for some index $k \leq q_s$ and some message m_k . Tokens are not handed to the adversary, so the request that accompanied this token to the services is the one the honest signer built, and therefore c^* carries the public key $pk_{e,k}$ of the ephemeral pair that the k -th signing query generated. The winning bundle satisfies $\Sigma_e.\text{Vf}(pk_{e,k}, m^*, \sigma^*) = 1$. The k -th signing query applied $sk_{e,k}$ to exactly two inputs, the subject inside the possession proof and the digest m_k , and the encoding of Figure 2 places these in separate domains, so σ^* is a signature on an input outside that set as soon as $m^* \neq$

m_k . That inequality holds because the winning condition requires that (id^*, m^*) was never stored while the k -th query stored (id^*, m_k) .

Game G_4 is G_3 with a guess k^* drawn uniformly from $[q_s]$ before the game starts and an abort if $k^* \neq k$. The guess is independent of the view of $\mathcal{A}$, so $\Pr[G_3] \leq q_s \Pr[G_4]$. We build $\mathcal{B}_3$ against Σ_e : it places its challenge key as pk_{e,k^*} , obtains the two signatures of that query from its signing oracle, generates the ephemeral pairs of the remaining queries itself, and outputs (m^*, σ^*) . Hence $\Pr[G_4] \leq \text{Adv}_{\Sigma_e}^{\text{uf}}(\mathcal{B}_3)$.

Collecting the four steps gives

$$\text{Adv}_{\text{SS}}^{\text{uf}}(\mathcal{A}) \leq \text{Adv}_{\Sigma_I}^{\text{uf}}(\mathcal{B}_1) + \text{Adv}_{\Sigma_{CA}}^{\text{uf}}(\mathcal{B}_2) + q_s \text{Adv}_{\Sigma_e}^{\text{uf}}(\mathcal{B}_3) + \text{Adv}_H^{\text{cr}}(\mathcal{B}_4),$$

which is the claimed bound. We note where each branch of the case analysis is used. The branch that ends in the ephemeral term is exactly the one an adversary takes when it pairs a certificate copied from the public log with an artifact signature of its own making and submits the result as a log entry, which the transparency component admits because it checks only that the signature is consistent with the verification material the entry carries.

A.2 Proof of Theorem 4.4

The first phase adversary $\mathcal{A}_1$ makes no query and outputs an empty state. At the quantum epoch the verifier trust state ts is fixed and $\mathcal{A}_2$ receives the secret keys $sk_{CA}, sk_{CT}, sk_{TS}, sk_L$ of the four schemes in the break set. Fix any target identity $id^* = (iss^*, sub^*)$ and message m^* , and any time $t^* < t_Q$ that the verification policy accepts.

The adversary proceeds as follows. It runs $\Sigma_e.\text{Kg}$ to obtain a pair (sk', pk') of its own, draws a serial number sn , and forms the certificate body $c^* = (sn, pk', iss^*, sub^*, t^*, t^* + \Delta_c)$. It computes $s^* = \Sigma_{CT}.\text{Sign}(sk_{CT}, (lid, t^*, c^*))$ and $\varsigma_{CA}^* = \Sigma_{CA}.\text{Sign}(sk_{CA}, (c^*, s^*))$, which gives the certificate $crt^* = (c^*, s^*, \varsigma_{CA}^*)$. It signs the message, $\sigma^* = \Sigma_e.\text{Sign}(sk', m^*)$, forms $u^* = (H(\sigma^*), t^*)$ and $\theta^* = (u^*, \Sigma_{TS}.\text{Sign}(sk_{TS}, u^*))$. It sets $b^* = (m^*, \sigma^*, crt^*)$, takes the one element leaf list $L^* = (H(0x00||b^*))$ with index $i^* = 0$ and empty inclusion proof, and computes $ck^* = (org, 1, \text{MTH}(L^*), t^*)$ and $\varsigma_L^* = \Sigma_L.\text{Sign}(sk_L, ck^*)$. It outputs the bundle assembled from these values.

Each check of SS.Vf in Figure 2 passes. The signed certificate timestamp verifies under pk_{CT} , the certificate verifies under pk_{CA} , the artifact signature verifies under the key that c^* carries, the timestamp token verifies under pk_{TS} , the checkpoint verifies under pk_L , and the inclusion proof verifies because MTH of a single leaf is that leaf. The identity in c^* equals id^* by construction, and t^* lies inside $[t^*, t^* + \Delta_c]$. The four public keys the verifier uses are the ones its trust state held at t_Q , and those are the keys whose secrets are in the break set. The identity id^* was never marked corrupted and the pair (id^*, m^*) was never stored, because no oracle was ever called, so the winning conditions hold and the advantage is one. The work is one key generation, four signatures and a constant number of hash evaluations, which is the cost of one honest issuance.

A.3 Proof of Theorem 4.5

Write $L^\dagger = (x_0, \dots, x_{E^\dagger-1})$ for the list of leaves the log holds at t_Q and $\Lambda^\dagger = \text{MTH}(L^\dagger)$ for the root that the verifier trust state carries. Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be an adversary that wins, and let its bundle contain a leaf preimage ρ^* that binds id^* and m^* , an index i^* and an inclusion proof pf^* accepted against $\Lambda^\dagger$. We construct a collision finder C that runs the whole experiment internally and distinguishes three cases.

The first case is $H(0x00\|\rho^\star) = x_{i^\star}$ together with $\rho^\star = \rho_{i^\star}$, where $\rho_{i^\star}$ is the preimage the log holds at that index. The leaves of $L^\dagger$ are exactly the entries the log accepted before t_Q , and during that whole period the services were honest and the adversary was the classical algorithm $\mathcal{A}_1$. We turn this case into an adversary $\mathcal{D}$ against Definition 4.1: $\mathcal{D}$ runs $\mathcal{A}_1$ against its own challenger, forwarding every oracle call, and at the end of the first phase it walks the leaves the log accepted, finds the index $i^\star$ and assembles from $\rho_{i^\star}$ together with the inclusion proof that the log can produce a bundle that the verifier accepts for the identity and the message that $\rho_{i^\star}$ binds. If that pair is an uncorrupted identity with an unsigned message, then $\mathcal{D}$ wins the classical game; and this is precisely the situation in which $\mathcal{A}$ wins the harvest-now-forge-later game through this case, since the leaf that its bundle uses is $\rho_{i^\star}$ itself and the winning conditions of the two games agree on the pair. Hence the probability of this case is at most $\text{Adv}_{\Pi}^{\text{uf}}(\mathcal{D})$. The quantum phase contributes nothing here, because the leaf was already fixed at t_Q .

The second case is $H(0x00\|\rho^\star) = x_{i^\star}$ with $\rho^\star \neq \rho_{i^\star}$. Then $0x00\|\rho^\star$ and $0x00\|\rho_{i^\star}$ are two distinct inputs with the same image, and C outputs them.

The third case is $y := H(0x00\|\rho^\star) \neq x_{i^\star}$. Verification of an inclusion proof against a tree of a fixed size and at a fixed index produces a chain of t applications of H whose left and right pattern is determined by the pair $(i^\star, E^\dagger)$ alone, so we may write the accepting computation as $y = a_0, a_1, \dots, a_t = \Lambda^\dagger$ with a_j obtained from a_{j-1} and the proof node p_j , and the honest computation of $\text{MTH}(L^\dagger)$ restricted to the same chain as $x_{i^\star} = b_0, b_1, \dots, b_t = \Lambda^\dagger$ with b_j obtained from b_{j-1} and the tree node u_j , both under the identical pattern. We have $a_0 \neq b_0$ and $a_t = b_t$, so there is a smallest index $j \geq 1$ with $a_j = b_j$. For that index the two arguments of H differ, because $a_{j-1} \neq b_{j-1}$, while the two images agree, and C outputs the two arguments.

The second and the third case therefore give a collision for H , and the first gives a classical forgery, so $\text{Adv}_{\Pi, \mathcal{B}}^{\text{hnl}}(\mathcal{A}) \leq \text{Adv}_{\Pi}^{\text{uf}}(\mathcal{D}) + \text{Adv}_H^{\text{cf}}(C)$. Neither term mentions the break set, and only the first mentions a signature scheme, which is what separates the two phases. For the numeric form, H is modeled as a quantum random oracle with $8n_h$ bit output; the quantum query complexity of finding a collision in a random function is $\Theta(2^{8n_h/3})$, and the matching upper bound on the success probability of a q query algorithm is $C(q+1)^3 2^{-8n_h}$ for an absolute constant C , which gives the stated inequality.

A.4 Proof of Theorem 5.2

Let $\mathcal{A}$ be a quantum adversary against the d instance one-time adaptive unforgeability of ROTS making q queries to H_m . We use three games.

Game G_0 is the experiment itself, with H_m a quantum random oracle. The challenger generates d instances of WOTS-TW under the public parameter P and the addresses $A_i = \text{kid}_i \| \text{tag}$ with kid_i drawn uniformly, hands $\mathcal{A}$ the public keys $pk_1, \dots, pk_d$, answers at most one signing query per instance, and declares $\mathcal{A}$ the winner when it returns $(i^\star, m^\star, (R^\star, \varsigma^\star))$ with $m^\star$ different from the message signed under instance $i^\star$ and with the verification of Definition 5.1 accepting.

Game G_1 differs in how signing queries are answered. Before the game starts the challenger draws $\mu_1, \dots, \mu_d$ uniformly from $\{0, 1\}^{8n}$. On the signing query for instance i with the adaptively

chosen message m_i , it draws R_i uniformly from $\{0, 1\}^{8n}$, reprograms H_m at the point $R_i \| pk_i \| m_i$ to the value μ_i , and answers with $(R_i, \text{WOTS-TW}.\text{Sign}(sk_i, \mu_i))$. The two games differ only in the reprogramming, and each reprogramming instruction places a uniformly drawn first component R_i from a set of size 2^{8n} in front of a second component $pk_i \| m_i$ that the adversary has already fixed, and maps it to a uniformly drawn value. This is the setting of the adaptive reprogramming bound with d instructions, so

$$|\Pr[G_0] - \Pr[G_1]| \leq \frac{3d}{2} \sqrt{\frac{q}{2^{8n}}}.$$

Game G_2 is G_1 with the additional rule that the adversary loses if its forgery satisfies $H_m(R^\star \| pk_{i^\star} \| m^\star) = \mu_{i^\star}$. Consider the predicate on the domain of H_m that marks an input x when x contains one of the d public keys in its middle component, say pk_i , and $H_m(x) = \mu_i$. Since the values $\mu_1, \dots, \mu_d$ are drawn uniformly and independently of the oracle outside the d reprogrammed points, and since a given input contains at most one public key, the predicate marks each input independently with probability 2^{-8n} . The forgery point is not one of the reprogrammed points, because $m^\star \neq m_{i^\star}$ forces $R^\star \| pk_{i^\star} \| m^\star \neq R_{i^\star} \| pk_{i^\star} \| m_{i^\star}$. An adversary that triggers the new rule therefore returns a marked input after q queries, and the generic quantum bound for finding a marked input of density λ after q queries, which is $8\lambda(q+1)^2$, gives

$$|\Pr[G_1] - \Pr[G_2]| \leq \frac{8(q+1)^2}{2^{8n}}.$$

It remains to bound $\Pr[G_2]$. We build an adversary $\mathcal{B}$ against the d instance non-adaptive unforgeability of WOTS-TW. In its first stage $\mathcal{B}$ draws $\mu_1, \dots, \mu_d$ uniformly and submits them as the list of messages to be signed, one per instance, which is permitted because that notion asks for the messages before the public keys are handed out. It receives the public parameter P , the public keys $pk_1, \dots, pk_d$ and the signatures $\varsigma_1, \dots, \varsigma_d$. It then simulates G_2 towards $\mathcal{A}$: it answers quantum queries to H_m with a $2q$ wise independent function, which is indistinguishable from a random oracle for a q query algorithm, patched by a classical list of reprogrammed points; it hands $\mathcal{A}$ the public keys; and on the signing query for instance i it draws R_i , adds the pair $(R_i \| pk_i \| m_i)$ to the patch list and returns (R_i, ς_i) . This is exactly the distribution of G_2 . When $\mathcal{A}$ outputs $(i^\star, m^\star, (R^\star, \varsigma^\star))$, the simulator computes $\mu^\star = H_m(R^\star \| pk_{i^\star} \| m^\star)$ and outputs $(i^\star, \mu^\star, \varsigma^\star)$. Whenever $\mathcal{A}$ wins in G_2 the verification of WOTS-TW accepts $\varsigma^\star$ on $\mu^\star$ under $pk_{i^\star}$, and $\mu^\star \neq \mu_{i^\star}$ by the rule of G_2 , so the output is a valid non-adaptive forgery. Hence $\Pr[G_2] \leq \text{Adv}_{\text{WOTS-TW}}^{d\text{-na}}(\mathcal{B})$, and collecting the three bounds proves the theorem. The simulator runs $\mathcal{A}$ once and evaluates a $2q$ wise independent function on each query, so its running time is that of $\mathcal{A}$ up to this overhead.

A.5 Proof of Corollary 5.3

The multi-instance non-adaptive bound for WOTS-TW with d instances, ℓ chains and Winternitz parameter w states that

$$\text{Adv}_{\text{WOTS-TW}}^{d\text{-na}}(\mathcal{B}) \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}_1) + \text{Adv}_{\text{Th}, d\ell w}^{\text{tcr}}(\mathcal{B}_2) + \text{Adv}_{\text{Th}, d\ell}^{\text{pre}}(\mathcal{B}_3) + w \text{Adv}_{\text{Th}, d\ell}^{\text{ud}}(\mathcal{B}_4),$$

where F is the pseudorandom function that expands the chain seeds and each of the three properties of the tweakable hash function is taken with distinct tweaks and with the stated number of targets. Substituting this into Theorem 5.2 gives the statement. The count of

targets follows the structure of the scheme: a target collision may be sought at any of the ℓw positions of any of the d instances, while a preimage challenge and an undetectability challenge sit at one position of each of the ℓ chains of each instance. The addresses of Definition 5.1 carry a 128 bit key identifier drawn independently for each instance, so all $d\ell w$ tweaks are distinct unless two identifiers collide, an event of probability at most $d^2 2^{-128}$ that we absorb into the first term.

A.6 Proof of Theorem 6.1

Let $\mathcal{A}$ be an adversary in $\text{Exp}_{\text{Latch}, \mathcal{A}}^{\text{uf}}$ whose output bundle parses as $(\rho^*, j^*, pf^*, hd^*, s^*)$.

Game G_0 is the experiment, and game G_1 adds the hash collision abort of Appendix A.1, which costs $\text{Adv}_H^{\text{cr}}(\mathcal{B}_3)$.

Game G_2 adds an abort if hd^* was never an argument of Σ_A . Sign inside an execution of Issue. As in Appendix A.1, an adversary $\mathcal{B}_2$ that embeds a challenge key as pk_A and obtains each epoch signature from its signing oracle outputs (hd^*, s^*) on the abort event, so $|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{\Sigma_A}^{\text{uf}}(\mathcal{B}_2)$.

In G_2 the head hd^* is one the service produced, so the root ρ_e inside it is the root of the epoch tree the service built from the entries it admitted in that epoch. The bundle carries an inclusion proof of $H(0x00\|\rho^*)$ at index j^* accepted against ρ_e . The chain argument of Appendix A.3 applies unchanged with $L^\dagger$ replaced by the leaf list of that epoch: in the absence of a collision, the leaf value at index j^* equals $H(0x00\|\rho^*)$ and the preimage ρ^* equals the entry ρ_{j^*} the service admitted there. Verification also checks that the identity field of ρ^* is id^* and that its digest field is m^* , so the service admitted an entry for id^* carrying m^* , which by the admission rule of Figure 5 required an identity token for id^* that verified under pk_I .

Game G_3 adds an abort if that token was never an output of Auth, which as in Appendix A.1 costs $\text{Adv}_{\Sigma_I}^{\text{uf}}(\mathcal{B}_1)$.

In G_3 the token was produced for id^* , and since id^* is never corrupted it was produced inside a call $O_{\text{sig}}(id^*, m_k)$. Tokens are not handed to the adversary, so the entry that reached the service under that token is the one the honest signer built, whose digest field is m_k . We concluded above that this entry is ρ^* and that its digest field is m^* , so $m^* = m_k$ and the pair (id^*, m^*) was stored, which contradicts the winning condition. Hence $\Pr[G_3] = 0$ and collecting the three steps gives the bound.

The comparison with Theorem 4.2 is now visible in the proof and not only in the statement. The step that produced the ephemeral term there was the one in which an honest certificate could be combined with an adversarial artifact signature, and it exists because the transparency component admits an entry after checking only that the entry is internally consistent. In Latch the certificate and the entry are the same object and the admission rule requires a token, so no such combination is available and the case analysis ends one step earlier.

A.7 Proof of Theorem 6.2

The first assertion is the content of games G_1 and G_2 of Appendix A.6: if $\mathcal{V}$ accepts under head hd_e and no collision for H occurs, then the leaf $H(0x00\|\rho)$ is the value at index j of the epoch tree whose root is inside hd_e , and ρ is the entry admitted at that index. A monitor that reads the entries under the published heads therefore sees ρ

itself, which contains the issuer, the subject, the one-time public key, the randomizer, the digest and the one-time signature.

For the second assertion, suppose the entry carries a one-time signature ς_{ots} that verifies under pk on the digest $\mu = H_m(R\|pk\|iss\|sub\|m)$, and suppose the party that produced it does not hold the corresponding secret. Over the whole game the honest signers create at most q_s one-time instances, each of which is used to sign one digest. An adversary that produces an accepting signature under one of those public keys on a digest that was not the one signed is, by definition, an adversary against the q_s instance one-time adaptive unforgeability of ROTS, and a reduction that generates the instances through its own challenger and returns the offending triple has exactly the advantage of that adversary. An adversary that produces an accepting signature under a public key of its own choosing does hold the secret for it, which is the assertion. Hence the probability that an accepted entry carries a one-time signature that its holder did not produce is at most $\text{Adv}_{\text{ROTS}}^{q_s\text{-euf1}}(\mathcal{B}_2)$.

The last assertion is immediate. A verifier that also checks hd_e against a view of the tree of heads that a set of witnesses has cosigned accepts only heads that appear in that view, so a service that shows one head to the verifier and a different one to the monitors would have to obtain cosignatures on two inconsistent views.

A.8 Proof of Corollary 6.3

In anchored mode the bundle carries the entry ρ , its index and inclusion proof inside the epoch tree, the head hd_e , and an inclusion proof of $H(0x00\|hd_e)$ in the tree of heads, and verification checks the second proof against the root $\Lambda^\dagger$ that the trust state fixed at t_Q . The composition of the two proofs is an inclusion proof of $H(0x00\|\rho)$ in a two level tree whose root is $\Lambda^\dagger$, and ρ binds the identity and the message, so the hypotheses of Theorem 4.5 are met with the tree hash of RFC 6962 and $n_h = 32$; a collision extracted from either level is a collision for H .

It remains to evaluate the first term of that theorem for the anchored verifier. The classical adversary $\mathcal{D}$ it produces has to bring an entry for an uncorrupted identity into the log before t_Q , and by the admission rule of Figure 5 the service writes such an entry only against an identity token that verifies under pk_I . Repeating game G_3 of Appendix A.6 shows that such a token is either a forgery against Σ_I or one produced inside a call $O_{\text{sig}}(id^*, m_k)$, and in the latter case the entry that reached the service is the honest one, whose message field is the stored m_k , so the pair is stored and $\mathcal{D}$ does not win. Hence $\text{Adv}_{\text{Latch-A}}^{\text{uf}}(\mathcal{D}) \leq \text{Adv}_{\Sigma_I}^{\text{uf}}(\mathcal{D}')$, and the anchor scheme does not appear because the anchored verifier never checks its signature.

A.9 Concrete parameters

At the first NIST security category we use $n = 16$ byte chains and $w = 16$, so that $\ell_1 = \lceil 8 \cdot 16/4 \rceil = 32$, $\ell_2 = \lfloor \log_2(32 \cdot 15)/4 \rfloor + 1 = 3$ and $\ell = 35$, and a one-time signature is 560 bytes; these are the values of the 128 bit SHA-2 parameter sets of SLH-DSA. At the third category we use $n = 24$, which gives $\ell_1 = 48$, $\ell_2 = 3$, $\ell = 51$ and a signature of 1224 bytes, and at the fifth category $n = 32$, $\ell = 67$ and 2144 bytes. The tweakable hash function is $\text{Th}(P, A, M) = \text{trunc}_n(\text{SHA-256}(P\|A\|M))$ with a 22 byte address A laid out as a 16 byte key identifier, a one byte type, a two byte chain index, a two byte position and one reserved byte, so that a chain step at $n = 16$ hashes 54 bytes and fits in

a single compression call. The tree that commits the entries and the tree that commits the heads use the full 32 byte output of SHA-256 with the leaf and node prefixes of RFC 6962. The anchor scheme is ML-DSA-44 at the first category and ML-DSA-65 at the third. The epoch batch is $B = 1024$ and the epoch length is the batching interval of the transparency service, which is on the order of ten seconds.

A.10 Randomizer length and concrete terms

Let ROTS_r denote the scheme of Definition 5.1 with the randomizer drawn from $\{0,1\}^{8r}$ for some $r \geq n$, so that ROTS_n is ROTS . The proof of Theorem 5.2 in Appendix A.4 applies to ROTS_r with a single change. In the step from G_0 to G_1 each reprogramming instruction places a first component drawn uniformly from a set of size 2^{8r} , so the adaptive reprogramming bound gives $\frac{3d}{2}\sqrt{q/2^{8r}}$. The step from G_1 to G_2 marks an input by the value that H_m takes on it, and the density 2^{-8n} of that predicate is fixed by the digest length and not by the shape of the input, while the reduction to WOTS-TW never reads the randomizer. The other two terms are therefore unchanged and

$$\text{Adv}_{\text{ROTS}_r}^{d\text{-euf1}}(\mathcal{A}) \leq \text{Adv}_{\text{WOTS-TW}}^{d\text{-na}}(\mathcal{B}) + \frac{3d}{2}\sqrt{\frac{q}{2^{8r}}} + \frac{8(q+1)^2}{2^{8n}}.$$

The reprogramming term reaches one at $\log_2 q = 8r - 2\log_2(3d/2)$, which is $8r - 81.2$ for $d = 2^{40}$, and the digest term reaches one at $\log_2 q = (8n - 3)/2$ up to the negligible effect of the summand one in $q + 1$. These two expressions give the entries of Table 2. With $r = n$ the reprogramming thresholds are 46.8, 110.8 and 174.8 at $n = 16, 24$ and 32; with $r = 32$ the threshold is 174.8 at all three; and the digest thresholds are 62.5, 94.5 and 126.5. The reprogramming term is thus binding only at the first category with $r = n$, and elsewhere the digest term binds, which is the generic Grover bound for an $8n$ bit preimage that SLH-DSA carries at the same n . The randomizer is part of the request, so the leaf stores it and the bundle carries it, and moving from $r = n$ to $r = 32$ adds $32 - n$ bytes to both, which is 16 bytes at the first category, 8 at the third and none at the fifth. All measurements in Section 7 use $r = n$.

A.11 Cost accounting

All methods emit the same wire format, which is a sequence of length delimited fields with a variable length prefix, so that a field of k bytes costs $k + 1$ bytes below 128 and $k + 2$ bytes below 16384. This is the encoding shape of the deployed bundle format, and using one encoding for all methods means that the sizes we compare differ only in the cryptographic objects the methods carry. A certificate is encoded as the fields a code signing certificate profile requires, which are a serial number, an algorithm identifier, the subject public key, the issuer and subject of the identity, a validity window, an extended key usage, a subject key identifier and an authority key identifier, followed by the signed certificate timestamp and the authority signature. A timestamp token is encoded as a policy identifier, the hash of the artifact signature, a time and a nonce, followed by the authority signature. A log entry is encoded as a kind, an interface version, a hash algorithm identifier, the artifact digest, the artifact signature and the verification material. Inclusion proofs against a log of 2^{27} entries materialize the co-path only, which reproduces the verifier cost exactly while avoiding a tree of several gigabytes; the anchored variants materialize the full tree of 2^{20} epoch heads and place the measured epoch at its midpoint.

Table 6: The batch sweep of Latch at the first category, with the epoch batch size B , the bundle size in bytes and the service time per signing event in milliseconds.

B	Bundle	Service (ms)
1	3244	0.125
4	3308	0.045
16	3373	0.031
64	3437	0.025
256	3501	0.025
1024	3565	0.025
4096	3629	0.026
16384	3693	0.026

Table 7: Hash calls per signing event in Latch for the signer, the service and the verifier, measured as averages over one epoch and predicted by the formulas of Section 6.4 at $B = 1024$, for each NIST category with chain length n in bytes and ℓ chains.

Cat.	n	ℓ	Signer		Service		Verifier	
			meas.	pred.	meas.	pred.	meas.	pred.
1	16	35	855.5	842.5	270.5	265.5	276.2	275.5
3	24	51	1244.0	1198.5	394.0	385.5	401.8	395.5
5	32	67	1641.8	1573.5	508.2	505.5	515.9	515.5

Table 8: The Winternitz sweep of Latch for epochs of 64 entries, with the Winternitz parameter w , the chain length n in bytes, the number ℓ of chains, the one-time signature and bundle sizes in bytes, and the signer and verifier times in milliseconds. The rows with $n = 16$ use the anchor scheme of the first category and those with $n = 24$ the anchor scheme of the third.

w	n	ℓ	Signature	Bundle	Signer	Verifier
4	16	68	1088	3965	0.029	0.048
16	16	35	560	3437	0.045	0.053
256	16	18	288	3165	0.299	0.138
4	24	101	2424	6206	0.049	0.073
16	24	51	1224	5006	0.086	0.083
256	24	26	624	4406	0.641	0.250

A.12 Full experimental data

Table 6 gives the batch sweep of Section 7 in numbers, Table 7 the hash call profile of Latch at the three security categories against the analytic model, Table 8 the Winternitz sweep, and Table 9 the dependence of the bundle on the size of the log.

A.13 The harvest-now-forge-later experiment

The experiment of Table 5 first builds, for each construction, an honest history of eight epochs with sixteen entries each for an identity other than the victim, with signing times that end before the claimed signing time $t^* = t_Q - 86400$ seconds. The adversary then holds the secret keys of all services and runs issuance itself on the victim identity, a digest of its choice and the time t^* , which is the attack of Theorem 4.4 carried out with the code of the honest services.

Table 9: Bundle size in bytes against the number N of log entries at the first category, for an epoch or batch of eight entries. The direct ML-DSA-44 substitution carries an inclusion proof against the whole log, while Latch and batched issuance carry paths inside an epoch or a batch.

$\log_2 N$	Latch	Batched (MTC)	ML-DSA-44
10	3340	11797	14115
14	3340	11797	14243
18	3340	11797	14371
22	3340	11797	14499
26	3340	11797	14627
32	3340	11797	14819

Against Sigstore under ML-DSA-44 this yields a certificate, a signed certificate timestamp, a timestamp token and a checkpoint that all

verify under the trust root; against batched issuance it yields the three signed batch heads; and against self-contained Latch it yields one signed epoch head. All three bundles are accepted, and the cost is that of one issuance. In the anchored constructions the tree of heads holds 4104 heads when the verifier pins its root at t_Q , which are 4096 preloaded heads followed by the eight honest epochs. The adversary appends its own head at index 4104 and attaches the inclusion path of that head in the extended tree. A verifier that holds the pinned root rejects, because the path recomputes the root of a tree of 4105 heads, and a path that the pinned root accepts would yield a collision by the argument of Appendix A.3. A verifier that refreshes its root after t_Q obtains the root of the extended tree and accepts the same bundle. The anchored forgeries cost more than the others because the adversary computes the path in the extended tree, once for Latch-A and once for each of the three service trees of batched issuance, which is consistent with the factor of about three between their costs.